Python OOP and Core Python Programming Concepts

A Guide to Data Types, Variables, Modifiers, Operators, Decisions, and Loops

Name: HENG Limhong
ID: e20241185
Group: I3-AMS-A

Nine Topics

- Topic 1: Basic Data Types
- Topic 2: Variables and Attributes
- Topic 3: Modifiers and Access Control (Part 1)
- Topic 4: Modifiers and OOP Features (Part 2)
- Topic 5: Operators (Part 1)
- Topic 6: Operators (Part 2)
- Topic 7: Decision-Making Statements
- Topic 8: Decision-Making (Part 2)
- Topic 9: Loops and Loop Control

Built-in Types

- int: whole numbers, e.g. 20
- float: decimals, e.g. 3.75
- bool: True or False
- str: text, e.g. "Ana"
- In Python every value is an object with a type

```
print(type(20).__name__)
print(type(3.75).__name__)
print(type(True).__name__)
print(type("Ana").__name__)
```

```
Output
int
float
bool
str
```

Java: int, double, boolean are primitives; Python has no primitives.

Collections and Literals

- list [ ]: ordered, changeable
- tuple ( ): ordered, fixed
- dict { }: key-value pairs
- set { }: unique values, no order
- Literal = a fixed value written in code; variable = a name that refers to a value

```
scores = [90, 85]       # list
point = (2, 3)          # tuple
info = {"id": 1}        # dict
tags = {"a", "b"}       # set
x = 10   # x variable, 10 literal
flag = None             # None literal
```

Numeric, string, Boolean, None and collection literals.

Student Records

- A list of dictionaries stores complete records
- Each dict holds name (str), age (int), score (float)

```
students = [
  {"name": "Ana", "age": 20, "score": 88.5},
  {"name": "Ben", "age": 21, "score": 72.0},
]
for s in students:
    print(s["name"], s["age"], s["score"])
```

```
Output
Ana 20 88.5
Ben 21 72.0
```

Summary: types, collections, literals, records.

Student Class

- Local variable: created inside a method, only usable there
- Instance attribute: self.name, belongs to one object
- Class attribute: defined in the class body, shared

```
class Student:
  school = "Greenfield"
  def __init__(self, name, age):
    self.name = name
    self.age = age
  def intro(self):
    hi = "Hi"   # local
    return f"{hi}, I am {self.name}"
a = Student("Ana", 20)
b = Student("Ben", 21)
print(a.name, b.name, a.school == b.school)
print(a.intro())
```

```
Output
Ana Ben True
Hi, I am Ana
```

Java: fields are declared in the class; Python creates them on self.

Local vs Instance vs Class

Kind	Created in	Belongs to	Access
Local variable	Inside a method	That call only	Inside the method
Instance attribute	self.x in __init__	One object	obj.x
Class attribute	Class body	The class (shared)	Class.x or obj.x

Summary: where a name lives decides who can see it.

Naming Conventions

- name: public
- _name: non-public by convention, for internal use
- __name: name mangling, becomes _Class__name
- Python does not enforce private access like Java
- Underscores are not security
- @staticmethod: no self or cls
- @classmethod: receives cls

Java has public, protected and private keywords enforced by the compiler.

BankAccount

- holder: public
- _transactions: non-public by convention
- __balance: name-mangled
- is_valid_amount: static method
- bank_info: class method

```
class BankAccount:
  bank_name = "PyBank"
  def __init__(self, holder, balance):
    self.holder = holder
    self._transactions = []
    self.__balance = balance
  @staticmethod
  def is_valid_amount(x):
    return x > 0
  @classmethod
  def bank_info(cls):
    return cls.bank_name
  def deposit(self, x):
    if self.is_valid_amount(x):
      self.__balance += x
  def get_balance(self):
    return self.__balance
```

```
Output
150
PyBank
150
```

The mangled name is still reachable.

Abstraction and Polymorphism

- ABC and @abstractmethod force subclasses to implement area()
- Circle and Rectangle override area()
- Same call, different behavior = polymorphism
- Shape() itself cannot be created

```
from abc import ABC, abstractmethod
import math
class Shape(ABC):
  @abstractmethod
  def area(self): ...
class Circle(Shape):
  def __init__(self, r): self.r = r
  def area(self):
    return math.pi * self.r ** 2
class Rectangle(Shape):
  def __init__(self, w, h):
    self.w, self.h = w, h
  def area(self):
    return self.w * self.h
for s in [Circle(2), Rectangle(3, 4)]:
  print(type(s).__name__, f"{s.area():.2f}")
```

```
Output
Circle 12.57
Rectangle 12.00
```

Java: abstract class and interface.

Encapsulation, property, final

- Encapsulation: data and methods together; methods validate changes
- Different from strict access control
- @property: call a method like an attribute
- typing.final: a hint for type checkers; Python does not enforce it at runtime

```
from typing import final
@final
class Temperature:
  def __init__(self, c):
    self._c = c
  @property
  def fahrenheit(self):
    return self._c * 9 / 5 + 32
print(Temperature(100).fahrenheit)
```

```
Output
212.0
```

Summary: abstraction, inheritance, overriding, encapsulation.

Arithmetic, Comparison, Bitwise

- / true division, // floor division, % remainder, ** power
- == != > < >= <= give True or False
- & | ^ ~ << >> work on integer bits

5 = 101 and 3 = 011 in binary.

```
print(7 / 2, 7 // 2, 7 % 2, 2 ** 3)
print(7 == 7, 7 != 7, 7 > 2)
print(5 & 3, 5 | 3, 5 ^ 3)
print(~5, 5 << 1, 5 >> 1)
```

```
Output
3.5 3 1 8
True False True
1 7 6
-6 10 2
```

Grades and Eligibility

- and: both true; or: either true; not: reverses
- Award needs average of 75+ and no score below 60

```
scores = [80, 70, 90]
average = sum(scores) / len(scores)
passed = average >= 50
award = average >= 75 and min(scores) >= 60
print(average, passed, award)
```

```
Output
80.0 True True
```

Summary: arithmetic, comparison, bitwise, logical.

Assignment and Special Operators

- = += -= *= /= //= %= **= update a variable
- is checks the same object; == checks equal values
- in / not in test membership
- x if condition else y

Updating a student record.

```
score = 80
score += 5
subjects = ["math", "art"]
print("math" in subjects)
a = [1, 2]; b = a; c = [1, 2]
print(a is b, a is c, a == c)
status = "Pass" if score >= 50 else "Fail"
print(score, status)
```

```
Output
True
True False True
85 Pass
```

Precedence and Associativity

- Higher precedence runs first: ** then * / then + -
- Use parentheses to be clear
- ** is right-associative; - is left-associative
- and runs before or

Summary: assignment, identity, membership, precedence.

```
print(2 + 3 * 4)
print((2 + 3) * 4)
print(2 ** 3 ** 2)
print(10 - 4 - 3)
print(True or False and False)
```

```
Output
14
20
512
3
True
```

if, if-else, if-elif-else

- if runs code when a condition is True
- else is the alternative
- elif checks several conditions in order; the first match wins
- Grades: A 90+, B 80+, C 70+, D 50+, F below 50, Invalid outside 0-100

```
def grade(score):
  if score < 0 or score > 100:
    return "Invalid"
  elif score >= 90: return "A"
  elif score >= 80: return "B"
  elif score >= 70: return "C"
  elif score >= 50: return "D"
  else: return "F"
for s in (95, 82, 45, 120):
  print(s, grade(s))
```

```
Output
95 A
82 B
45 F
120 Invalid
```

Pass = D or better. Summary: order of conditions matters.

Nested if, match-case, Conditional Expression

- Nested if: a second check inside the first
- match-case (Python 3.10+) picks one branch; case _ is the default
- Java: value = cond ? v1 : v2;
- Python: value = v1 if cond else v2

Use match for many fixed values; if-elif for ranges.

```
age = 20
has_id = True
if age >= 18:
  if has_id:
    print("Entry allowed")
choice = "2"
match choice:
  case "1": print("View")
  case "2": print("Add")
  case _: print("Invalid")
label = "Adult" if age >= 18 else "Minor"
print(label)
```

```
Output
Entry allowed
Add
Adult
```

Student Management Menu

- 1 View, 2 Add, 3 Search, 4 Exit
- match-case selects the action

```
students = ["Ana", "Ben"]
choice, name = "3", "Ben"
match choice:
  case "1": print(students)
  case "2": students.append(name)
  case "3":
    print("Found" if name in students
          else "Not found")
  case "4": print("Goodbye")
  case _: print("Invalid option")
```

```
Output
Found
```

Summary: nested, match-case, conditional expressions.

while, for, break, continue, pass

- while repeats while a condition is True
- for iterates over sequences; range(n) gives 0..n-1
- No native do-while: use while True and break
- break exits the loop; continue skips one pass; pass does nothing

Java has do-while; Python does not.

```
n = 3
while n > 0:
  print(n); n -= 1
for i in range(2):
  print("i =", i)
while True:
  print("runs once")
  break
```

```
Output
3
2
1
i = 0
i = 1
runs once
```

Processing Student Objects

- Skip scores below 50 with continue
- Stop searching with break

```
class Student:
  def __init__(self, name, score):
    self.name, self.score = name, score
students = [Student("Ana", 88),
  Student("Ben", 45), Student("Cara", 92)]
total = count = 0
for s in students:
  if s.score < 50: continue
  total += s.score; count += 1
print(total / count)
for s in students:
  if s.name == "Cara":
    print("Found", s.name); break
```

```
Output
90.0
Found Cara
```

Average ignores Ben (45).

Main Concepts

- Topic 1-2: types, collections, attributes
- Topic 3-4: naming conventions, decorators, abstraction, encapsulation
- Topic 5-6: operators, identity, precedence
- Topic 7-8: if, match-case, conditional expressions
- Topic 9: while, for, break, continue, pass